

Спецификация MVP: Приложение "Manather"

Документ функциональных и технических требований для разработчика

Целевая платформа:	macOS (14.0 Sonoma и новее)	Тип проекта:	Локальное десктопное приложение
Технологический стек:	Swift, SwiftUI, SwiftData	Стадия:	Разработка MVP (Версия 1.0)

1. Общее описание и концепция MVP

Manather — это интеллектуальная десктопная галерея для разработчиков, дизайнеров и ИИ-энтузиастов, предназначенная для упорядоченного хранения визуальных референсов, скриншотов интерфейсов и сопутствующих метаданных (включая промпты генерации и текстовые заметки).

Главная цель MVP: Создать стабильное, легковесное локальное приложение с фокусом на фронтенд-составляющую. На данном этапе полностью исключаются сложные бэкенд-системы, автоматическое вычисление палитр и синхронизация. Вся работа происходит локально с файловой системой macOS и базой данных SwiftData.

2. Структура интерфейса (UI Layout)

Интерфейс приложения реализуется на базе стандартного паттерна macOS с тремя панелями с помощью `NavigationSplitView`:

Панель	Элементы интерфейса	Функционал на этапе MVP
Левая (Sidebar)	Список категорий: "Все элементы", "Корзина"	Фильтрация центральной сетки. Элементы в корзине имеют флаг удаленных.
Центральная (Content)	Поисковая строка, адаптивная сетка ассетов (LazyVGrid), ползунок масштаба, кнопка "+"	Отображение превью картинок в виде аккуратных карточек. Поиск по названию. Локальный импорт файлов.
Правая (Inspector)	Панель метаданных (переключается кнопкой в верхнем тулбаре)	Отображение и редактирование текстовых полей выделенного элемента. Автоматическое сохранение изменений.

3. Модель данных (SwiftData Architecture)

Локальное хранилище данных реализуется через фреймворк SwiftData. Для MVP требуется только одна ключевая модель — `AssetItem`.

Важное архитектурное правило для работы с файлами:

Приложение не должно хранить бинарные данные изображений (Data/Blob) прямо в базе SwiftData, чтобы избежать деградации производительности. При импорте файл физически копируется в изолированную локальную директорию приложения (Application Support или Documents), а в базе данных сохраняется только относительный путь (Relative Path).

Код структуры модели:

```
import Foundation
import SwiftData

@Model
final class AssetItem {
    @Attribute(.unique) var id: UUID
    var title: String
    var relativeFilePath: String // Путь к скопированному файлу внутри песочницы
    var sourceURL: String        // Ссылка на источник/сайт
    var prompt: String           // Текст ИИ-промпта
    var notes: String            // Пользовательские заметки
    var isTrash: Bool            // Флаг нахождения в корзине
    var dateAdded: Date          // Дата добавления для сортировки

    init(title: String, relativeFilePath: String, sourceURL: String = "", prompt: String = "", notes: String = "") {
        self.id = UUID()
        self.title = title
        self.relativeFilePath = relativeFilePath
        self.sourceURL = sourceURL
        self.prompt = prompt
        self.notes = notes
        self.isTrash = false
        self.dateAdded = Date()
    }
}
```

4. Функциональные требования

4.1. Импорт и управление файлами

- **Импорт через диалоговое окно:** Кнопка "+" в тулбаре вызывает системный `fileImporter`, позволяющий выбрать графический файл (поддержка PNG, JPG, WebP).
- **Поддержка Drag & Drop:** Реализовать возможность перетаскивания файлов из Finder напрямую в область центральной сетки. Файл автоматически копируется в песочницу приложения и регистрируется в SwiftData.
- **Авто-наименование:** По умолчанию элементу присваивается имя исходного файла (без расширения). Пользователь может изменить его в инспекторе.

4.2. Сетка просмотра (Gallery Grid)

- Отображение карточек с фиксированным соотношением сторон или квадратным кадрированием для сохранения чистоты сетки.

- Нижний или верхний слайдер регулировки масштаба (изменение количества колонок в LazyVGrid от 2 до 6).
- Динамический поиск: при вводе текста в поле поиска сетка мгновенно фильтрует элементы по совпадению в title или prompt.

4.3. Инспектор метаданных (Правая панель)

- Панель открывается при клике на любую карточку в сетке. Если выбор пуст — панель показывает заглушку "Выберите элемент".
- Содержит редактируемые текстовые поля:
 - TextField для названия (Title)
 - TextField для URL-источника (Source URL)
 - TextEditor для ИИ-промпта (Prompt) с кнопкой "Скопировать промпт" в буфер обмена
 - TextEditor для личных заметок (Notes)
- Все изменения в полях сохраняются в базу данных автоматически по мере ввода (интеграция со SwiftData через привязку состояний).

4.4. Жизненный цикл элемента (Удаление)

- Контекстное меню по правому клику мыши на карточку: пункт "Удалить в корзину".
- Действие переводит флаг `isTrash = true`, элемент исчезает из раздела "Все элементы" и появляется в разделе "Корзина".
- Внутри раздела "Корзина" контекстное меню предлагает два действия: "Восстановить" (`isTrash = false`) и "Удалить навсегда" (полное удаление записи из SwiftData и физическое удаление файла из папки приложения).

5. Исключено из скоупа MVP (В бэклог будущих версий)

Для обеспечения быстрой сборки прототипа следующие фичи **НЕ делаются** в первой версии:

1. Интеграция с ИИ-моделями и кнопка "Generate variation". На этапе MVP поля для промптов заполняются исключительно вручную методом копипаста.
2. Автоматический сбор цветовой палитры и инструмент "Пипетка".
3. Поддержка видеофайлов и интерактивных веб-компонентов (только статические изображения).
4. Облачная синхронизация (CloudKit) и работа с аккаунтами.

6. Критерии приемки (Acceptance Criteria)

- Приложение запускается локально на macOS без ошибок компиляции.
- Перетаскивание картинки из Finder создает новую плитку в галерее; после перезапуска приложения картинка остается на месте.
- Текст, вписанный в поле Prompt в правой панели, успешно сохраняется и находится через строку поиска.
- Интерфейс строго следует гайдлайнам macOS (отзывчивые панели, нативные шрифты, корректная работа при сворачивании панелей).